

Low-Rank Expert Rewriting: Imperative Reference vs Declarative BrainSurgery

The same declarative pattern extends from structural MoE upcycling to representation-level expert compression.

A. Reference Python Script

```
for layer in range(16):
    for proj in ("gate_proj", "up_proj", "down_proj"):
        expert0_key = f"model.layers.{layer}.mlp..."
        expert1_key = f"model.layers.{layer}.mlp..."
        delta = source[expert1_key] - source[expert0_key]
        approx_delta = reconstruct_phlora_rank(
            delta,
            rank,
            cache=cache,
            cache_key=expert1_key,
            ...
        )
    out[expert1_key] =
        source[expert0_key] + approx_delta
```

Low-rank approximation logic is handwritten inside loops and tensor mutation code.

B. BrainSurgery YAML Plan

```
transforms:
  - copy: { from: expert_1, to: expert_1.delta }
  - subtract_:
      from: model:...experts.0...
      to:  model:...experts.1...delta...
  - phlora:
      target:  model:...experts.1...delta...
      target_a: model:...phlora_a.weight
      target_b: model:...phlora_b.weight
      rank: 64
  - delete: { target: model:...experts.1.weight }
  - assert:
      shape: { of: model:...phlora_a.weight, is: [64, ...] }
  - assert:
      not: { exists: model:...experts.1.weight }
```

Compression, cleanup, and safety checks are expressed directly in the checkpoint plan.

C. Built-In Validation

```
inputs:
  - yaml::olmo_1b_0724_hf_low_rank_expert_r64_demo
  - ref::olmo_1b_0724_hf_low_rank_expert_r64_reference

transforms:
  - diff: { mode: aliases, left_alias: ref,
            right_alias: yaml }
```

Missing on left:
(none)

Missing on right:
(none)

Differing:
(none)

No differences found.

The low-rank rewrite is also validated against an independent reference.

Validated MoE checkpoint -> low-rank expert rewrite -> reference implementation -> BrainSurgery diff